Week 10 · Where the state goes: extraction and lifting

Technical Interaction Design · ITU · Autumn 2026

Built 15 October 2026 · commit `8de28e1`
Latest version: https://itu-tid.github.io/lecture-notes-pdf/Week-10.pdf

CONTENTS

Finding the Components

- Start by drawing boxes on the app you can already see
- Two boxes if either half would be useful on its own
- Extracting a component is moving JSX out and naming what it needs

Patterns of Component Communication

- Data goes down, as props
- Events come back up, as callbacks
- Passing `setTodos` down gives every row power over the whole list

Refactoring by Extracting Components

- Where the State Goes
- The same function, written twice

The `useRef` hook

- Read More

Component Extraction & Semantic React Architecture

- Universal Principles (CSS-Agnostic)
- The Problem: “Div Soup” & “Class Soup”
- The Solution: Semantic Component Extraction
- The Four Core Principles
- Step-by-Step Tutorial: Component Extraction
- Advanced Pattern: Polymorphic Components
- Recommended File Structure
- Checklist: When to Extract a Component
- Practice Exercise
- Additional Resources
- Key Takeaways

Finding the Components

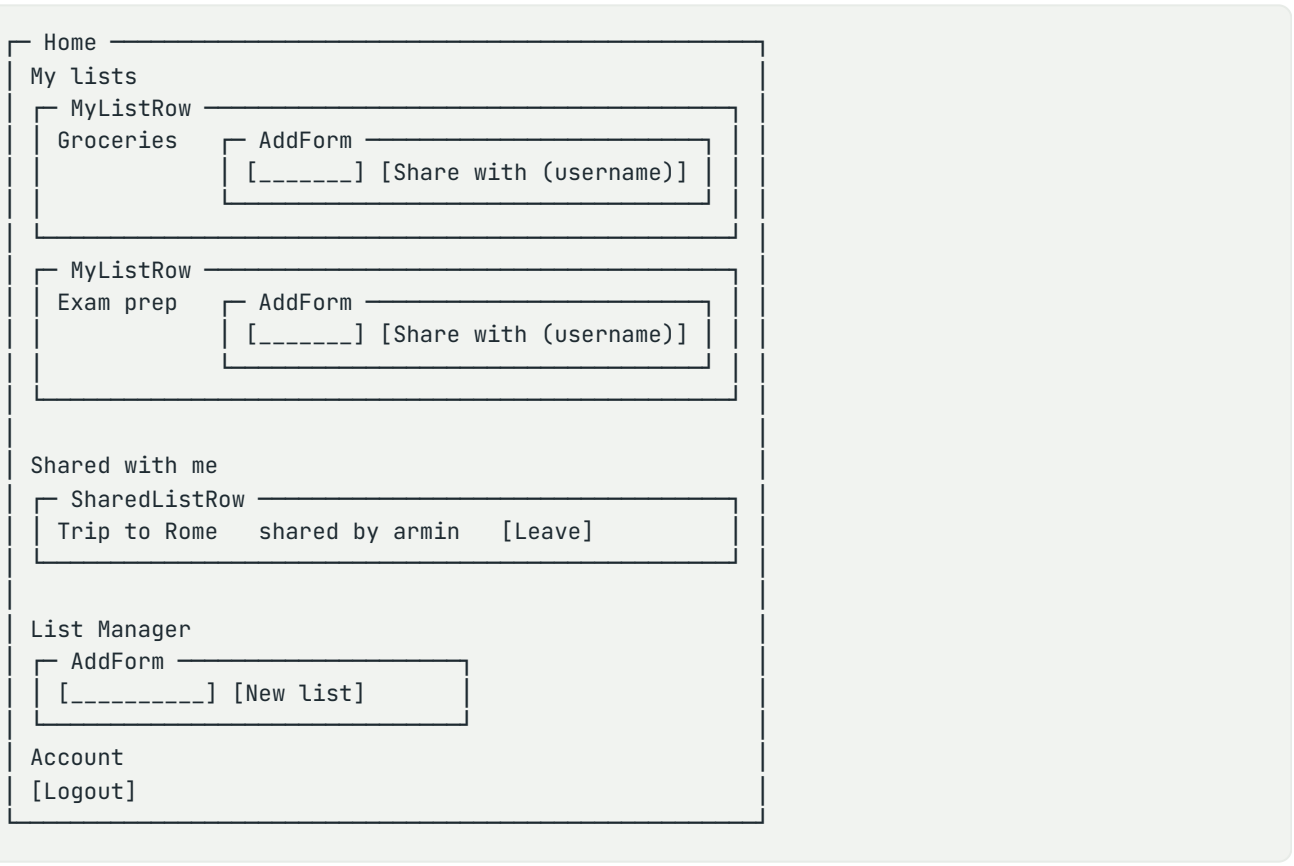
Source: Lectures/Technical/Structure/Finding-the-Components.md — something unclear? Open an issue, or send a pull request.

You know how to write a component. The harder question is which ones to write, and the answer is not obvious from the code, because at that point everything is already one big `return`.

So answer it on the screen instead. Draw boxes around the parts of the running interface, and name each box. It works both ways round: on a screen you have not built yet, and on one that already runs and has grown hard to read.

Start by drawing boxes on the app you can already see

Here is the home page of `todo-26`, as it runs after Pulling Out the Service Layer:



Boxes inside boxes give you the hierarchy for free:

```
Home
├── MyListRow        (one per list I own)
│   └── AddForm      (share it)
│       └── TextInput
├── SharedListRow    (one per list shared with me)
└── AddForm          (a new list)
    └── TextInput
```

The labels are your names for the boxes, not anything the user sees.

Notice what is *not* a box. The *My lists* heading, the *Logout* button and the *Leave* button are drawn but unnamed: each appears in exactly one place. **Draw a box when the same thing appears more than once.** `MyListRow` appears once per list, `SharedListRow` once per membership, and `AddForm` three times on this page alone, and once more on every list's page, for a new to-do.

That is the stopping rule, and you need one, because without it you can keep subdividing until every `<span>` is a component. *My lists* plus the rows under it would make a fine-sounding `MyLists` component, and it would be used exactly once. When you are unsure, leave it where it is: pulling a component out later is a five-minute job, and one you did not need is permanent clutter.

That is the whole exercise, and it takes two minutes on paper.

> **In todo-26:** `git checkout week-08-components`, and compare `Home.jsx` with the one in `week-08-service-layer`. The two rows moved out into `MyListRow.jsx` and `SharedListRow.jsx`; the rest of the page stayed where it was.

Something unclear in this section? Open an issue.

Two boxes if either half would be useful on its own

It is not about how many parts the thing has. `AddForm` is a text input *and* a button, and it is rightly one box: a button with nothing to submit is useless, and a field with no way to submit it is useless. Two parts, one job.

So the test is: **would either half be useful on its own? If yes, make them two. If neither means anything without the other, they are one.**

Then name it as a **thing, not an action**. A component is something on the screen, so it gets a noun: `MyListRow`, `SharedListRow`, `AddForm`. Functions that *do* something get the verb: `handleShare`, `onLeave`. If you find yourself writing `ShareList` as a component, you have named the button's job rather than the thing on the page.

A name also has to fit every place the thing is used. The form began, in Forms and Controlled Components, as `NewTodoForm`. Then it started creating lists, and then sharing them, and `NewTodoForm` with `buttonLabel="Share with (username)"` was a lie. It is `AddForm` now: a form that adds whatever its caller says.

- `MyListRow` — draws one of my lists. One box.
- `SharedListRow` — draws one list somebody shared with me. One box.
- `AddForm` — an input and a button, neither any use without the other. Still one box.

The common way to fail is welding the page's heading into a component that does not need it

`todo-26` still has one. `ToDoList` draws the to-dos and the form to add one, and, at the top, the page's title:

```
export default function ToDoList({ list }) {
  let h1Style = { color: "deeppink", backgroundColor: "white" };
  // …
  return (
    <>
      <h1 style={h1Style}>{list.get("name")}</h1>
      {/* … the to-dos, and the AddForm … */}
    </>
  );
}
```

Want the list without the big pink title, say as a preview of the first few to-dos on the home page? Want a share button next to the title, on the list's page? You cannot have either, because they were glued together before anybody asked.

Both halves would be useful alone, and that is the whole argument. It is the exact opposite of `AddForm`, where neither half would be. Notice too that the name gave nothing away: `ToDoList` sounds like one thing, and it is one thing plus a heading.

The heading belongs to the page. Split, `ListPage` says what goes on it, and `ToDoList` draws only the list:

```
// in ListPage
<>
  <h1>{list.get("name")}</h1>
  <ToDoList key={list.id} list={list} />
</>
```

Two boxes on the drawing, two components in the file. This one is left for you: in `todo-26` it is a two-line pull request.

Two other signals, both visible on the drawing rather than in the code:

It repeats. `MyListRow` appears once per list. Anything you would draw more than once is a component.

It matches the data. Notice that the two kinds of row are the two kinds of thing `App` loads: `lists`, rows of the `List` class, and `memberships`, rows of `ListMember`. One component per kind of row. When the component hierarchy and the data have the same shape, the code is usually easy; when they disagree, something is being forced. Try drawing one `ListRow` for both, and it fills up with `if (membership)`.

Something unclear in this section? Open an issue.

Extracting a component is moving JSX out and naming what it needs

Here is `Home` with the shared lists still inline:

```
{memberships.map((membership) => (
  <p key={membership.id}>
    <Link to={`/lists/${membership.get("list").id}`}>
      {membership.get("list").get("name")}
    </Link>{" "}
    shared by{" "}
    {membership.get("list").get("owner")?.get("username") ?? "someone"}{" "}
    <button onClick={() => onLeave(membership)}>Leave</button>
  </p>
))}
```

Cut the `<p>` out, and look at what it uses that it did not define itself: `membership`, from the `map`, and `onLeave`, from `Home`'s props. **Everything the moved JSX referenced from outside itself has to become a prop.**

```
export default function SharedListRow({ membership, onLeave }) {
  const list = membership.get("list");

  return (
    <p>
      <Link to={`/lists/${list.id}`}>{list.get("name")}</Link>{" "}
      shared by {list.get("owner")?.get("username") ?? "someone"}{" "}
      <button onClick={() => onLeave(membership)}>Leave</button>
    </p>
  );
}
```

The `key` stays behind. It is for the `map`, which tells one row from the next, not for the row:

```
{memberships.map((membership) => (
  <SharedListRow key={membership.id} membership={membership} onLeave={onLeave} />
))}
```

Once it had a name of its own, `membership.get("list")`, said three times, became one `const`. That kind of small tidying is easier once the code is in a box of its own, where you can see all of it at once.

A component around an input takes what the input takes

The same move works one level down. `AddForm` used to hold its `<input>` inline, as in Forms and Controlled Components:

```
function AddForm({ onAdd, buttonLabel = "Add New Task" }) {
  const [task, setTask] = useState("");
  // …
  return (
    <form onSubmit={onButtonClick}>
      <input type="text" value={task} onChange={(e) => setTask(e.target.value)} />
      <button type="submit" disabled={task.length === 0}>{buttonLabel}</button>
    </form>
  );
}
```

The `<input>` references the value and the change handler, so those become props:

```
export default function TextInput({ value, onChange }) {
  return (
    <input
      type="text"
      value={value}
      onChange={(event) => onChange(event.target.value)}
    />
  );
}
```

One line inside `TextInput` does more than move: `onChange={(event) => onChange(event.target.value)}` unwraps the event and passes up a plain string. Write `onChange={onChange}` instead and the parent gets an event object where it expected text. That is the commonest way an extraction like this goes wrong.

The state stays where it was; `TextInput` has none of its own:

```
<TextInput value={task} onChange={setTask} />
```

`useState` did not move into `TextInput`. If it had, `AddForm` would have no idea what was typed, and could never clear the field or submit the task. `TextInput` displays what it is given and reports what happened; it remembers nothing between renders.

Two things improved, and neither is about saving typing.

The parent stopped touching `event.target.value`. `TextInput` unwraps the event and hands up a plain string, so `AddForm` can pass `setTask` directly. The parent now works in the language of the app instead of the language of the DOM.

There is one place to change how inputs look. A `className`, a placeholder, anything you add later, lives here rather than in every screen that happens to need typing.

And look at the shape of it: `TextInput` takes the value it should show, and a way to report that something changed. Exactly what the raw `<input>` takes, under the same names. The pattern travelled up a level without changing, so the word travels with it: a **controlled component** is any component whose important state is held by its parent, exactly as a controlled input is an input whose value is held by the component around it.

Something unclear in this section? Open an issue.

Exam Questions

1. `AddForm` holds an input and a button and is one component. `ToDoList` holds the list's heading and its to-dos and should be two. What is the difference?

2. What is wrong with a component that renders a to-do heading *and* accepts arbitrary `children`?

3. `TextInput` holds no state of its own. Why not, and what would break if it did?

4. Why does todo-26's home page have a `SharedListRow` component, but no `MyLists` component?

References

- Thinking in React — the five steps, of which the first is the one worth keeping
- Importing and Exporting Components — once a component has its own name it usually wants its own file

Next: deciding *where the state lives* once you have pulled components out is its own problem, and extraction is what creates it: Refactoring by Extracting Components.

Patterns of Component Communication

Source: Lectures/Technical/React/Patterns-of-Component-Communication.md — something unclear? Open an issue, or send a pull request.

Components are arranged in a tree, and information has to travel along it. There are only two directions, and they use different mechanisms.

You have been writing both since week 2, without the names. The to-do list hands each row a `todo` and an `onRemove`; the row calls `onRemove` and knows nothing about what happens next. This note is that arrangement, named, and the shortcut that looks easier and is not.

Data goes down, as props

The parent hands the child what it needs to draw itself: for a row, the `todo` it should display. Data goes down.

Something unclear in this section? Open an issue.

Events come back up, as callbacks

The child does not know who its parent is — and should not. Imagine a button: it has no idea who put it there, and it would be a worse button if it did.

What it does have is a function it was handed. Calling it is how the child says *something happened*, without knowing or caring what that means:

```
function TodoList() {
  const [todos, setTodos] = useState([]);

  function handleRemove(id) {
    setTodos(todos.filter((t) => t.id !== id));
  }

  return (
    <ul>
      {todos.map((todo) => (
        <TodoItem
          key={todo.id}
          todo={todo}          // down: what to draw
          onRemove={handleRemove}   // up: a way to report a click
        />
      ))}
    </ul>
  );
}

function TodoItem({ todo, onRemove }) {
  return (
    <li>
      {todo.text}
      <button onClick={() => onRemove(todo.id)}>×</button>
    </li>
  );
}
```

Read the two directions:

- **Down, as props.** `TodoList` hands each row its `todo` — the data it needs to draw.
- **Up, as a callback.** `TodoItem` is handed `onRemove` and calls it. It does not know what happens next: it is not deleting anything, it is *reporting a click*.

`TodoItem` knows nothing about `filter`, about the array, or even that a list exists. Drop it into an app that archives items instead of deleting them, passing the same `onRemove`, and it works unchanged. That is what you get for keeping the arrow pointing one way.

Something unclear in this section? Open an issue.

Passing `setTodos` down gives every row power over the whole list

The tempting shortcut is to pass `setTodos` down and let the row remove itself:

```
<TodoItem todo={todo} todos={todos} setTodos={setTodos} />   // do not do this
```

It works. It also means every row now knows the whole list, and the shape of it, and how deletion is implemented — so changing any of those means changing the row too. You have swapped one small prop for three, and gained a component that only works in this app.

Hand down what the child needs to know, and a way to tell you something happened. Nothing else.

Next: once you extract components, two of them end up needing the same state, and neither owns it. That is *lifting state up*, and it is in Refactoring by Extracting Components — because extraction is what creates the problem in the first place.

Something unclear in this section? Open an issue.

Exam Questions

1. What are the two directions information travels between components, and what carries each?

2. `TodoItem` has a delete button but cannot delete anything. Why is that the right arrangement?

3. What goes wrong if you pass `setTodos` to every row instead of an `onRemove` callback?

Refactoring by Extracting Components

Source: Lectures/Technical/Structure/Refactoring-by-Extracting-Components.md — something unclear? Open an issue, or send a pull request.

Lesson: refactoring is critical for having a codebase that is maintainable and pleasant to work with

Challenge: Compare the following equivalent versions of `Home` in todo-26

- They have **exactly the same behavior**: the same page, the same links and buttons
- Which of them is easier to read?

Version 1 — `git checkout week-08-service-layer`

```
<h2>My lists</h2>
{lists.map((item) => (
  <div key={item.id}>
    <Link to={`/lists/${item.id}`}>{item.get("name")}</Link>
    <AddForm
      onAdd={(username) => onShare(item, username)}
      buttonLabel="Share with (username)"
    />
  </div>
))}

{memberships.length > 0 && <h2>Shared with me</h2>}
{memberships.map((membership) => (
  <p key={membership.id}>
    <Link to={`/lists/${membership.get("list").id}`}>
      {membership.get("list").get("name")}
    </Link>{" "}
    shared by{" "}
    {membership.get("list").get("owner")?.get("username") ?? "someone"}{" "}
    <button onClick={() => onLeave(membership)}>Leave</button>
  </p>
))}

<h3>List Manager</h3>
<AddForm onAdd={onAddList} buttonLabel="New list" />
<h3>Account</h3>
<button onClick={onLogout}>Logout</button>
```

Version 2 — `git checkout week-08-components`

```
<h2>My lists</h2>
{lists.map((list) => (
  <MyListRow key={list.id} list={list} onShare={onShare} />
))}

{memberships.length > 0 && <h2>Shared with me</h2>}
{memberships.map((membership) => (
  <SharedListRow key={membership.id} membership={membership} onLeave={onLeave} />
))}

<h3>List Manager</h3>
<AddForm onAdd={onAddList} buttonLabel="New list" />
<h3>Account</h3>
<button onClick={onLogout}>Logout</button>
```

- Version 2 reads like the page: two headings, two kinds of row, a form, a button. Everything in it is at the same abstraction level, that of the page.
- Version 1 is at every level at once
  - the page: headings, sections
  - one row: a link, a form, a button
  - the data: `membership.get("list").get("owner")?.get("username")`, a pointer followed through a pointer
  - the text between the elements: `{" "}`

The details did not go away. They moved to `SharedListRow`, where they are the whole point, and you read them only when the row is what you care about.

Principle: Code in a component should be at the same abstraction level

Advice: iterate. iterate. iterate

- there is no way to arrive to simple code without a lot of iteration

Advice 2: first make it work. then refactor it and make it nice

- you never get it right from the first time

Where the State Goes

Extraction has a consequence. Split one component into three and some of them will need to agree about the same piece of state — which now belongs to none of them. The two patterns from Component Communication (props down, callbacks up) are not enough on their own.

sibling to sibling by "*lifting the state*"

Sometimes, you want the state of two components to always change together.

To do it, you *lift the state up*:

- remove state from both of them
- move it to their closest common parent, and then
- pass it down to them via props

This is a common thing you will do when writing React

Example: in the TODO list application, ensuring that one can't start two tasks in two different lists

Sending a message from one component to the other

- also has to be done via the parent
- in our case, imagine that you want to move one of the tasks, from one list to the other
- the parent would have to know about all the lists and a given list will have to call a callback on the parent to decide to which list does one

To read for next time: Sharing state between components

- talks about lifting state with an example

Exam Questions

1. What does "lifting state up" mean in React?

2. Why is this principle important: *"Code in a component should be at the same abstraction level"*?

Something unclear in this section? Open an issue.

The same function, written twice

Components are not the only thing worth extracting. A common case: `const getCurrentUser = async () ⇒ …` defined once in `ProfileBar` and again in `Settings`, because both screens needed it and neither knew about the other.

Two copies of a function are two things to fix when the backend changes, and they drift silently — one gets a null check, the other does not. Pull it into a **custom hook** (`useCurrentUser`) and both screens call the same code.

The signal to watch for: you are about to write something you have a feeling you already wrote. You probably did.

Something unclear in this section? Open an issue.

The `useRef` hook

Source: Lectures/Technical/React/The-useRef-Hook.md — something unclear? Open an issue, or send a pull request.

Allows you to get hold of a DOM element and manipulate it

Example: Automatically focusing a given element on first render

```
import {useRef, useEffect} from 'react'

const InputRefExample = () => {

      const inputRef = useRef(null);

      useEffect(() => {
        inputRef.current?.focus()
      }, [])

      return(
        <div>
          <input ref={inputRef} />
          <button>
            Submit
          </button>
        </div>
      )
    }

export default inputRefExample
```

Canonical use of the `useRef` hook

1. Import `useRef`
2. Declare a ref object (i.e. `inputRef`) - initialize it with null, it's fine
3. Pass your object as the `ref` attribute to the desired DOM element
4. Refer to the DOM object by using the `current` property

Other Examples of manipulating the DOM

- scrolling to a given node
- getting the dimensions of a given DOM node
- keeping track of a timer (see the TODO list application)

Notes

Optional chaining to prevent errors

The optional chaining (`?.`) prevents errors if the element isn't mounted yet.

```
      useEffect(() => {
        inputRef.current?.focus()
      }, [])
```

The reference is persisted between renders

State changes of the referenced element are not observed by React

Read More

- useRef in the react documentation
- Nice example of controlling a video player component
  - you need to interact with the video element in the DOM
  - the DOM has already a powerful implementation of
  - get a reference to the relevant DOM element
  - call the `start` and `stop` methods

Exam Questions

1. What is the purpose of the useRef hook?

2. What does this code do and why use optional chaining?

```
const inputRef = useRef(null);

useEffect(() => {
  inputRef.current?.focus();
}, []);

return <input ref={inputRef} />;
```

Something unclear in this section? Open an issue.

Component Extraction & Semantic React Architecture

Source: Lectures/Technical/Structure/Component-Extraction-Guide.md — something unclear? Open an issue, or send a pull request.

> **On the examples below.** They are written in TypeScript with Tailwind class names, because they came from a different course. This course is JavaScript and MUI. Read them for the *decisions* — when a block deserves a name, when repetition earns a component — and ignore the syntax around them; nothing here depends on the stack.

Universal Principles (CSS-Agnostic)

These principles apply whether you use Tailwind, CSS Modules, styled-components, or plain CSS.

The Problem: "Div Soup" & "Class Soup"

Bad Example (Tailwind)

```
<div className="mx-auto flex w-full max-w-xl flex-col items-start gap-6 px-4 py-6">
  <section className="flex w-full flex-col items-center gap-2 p-6">
    <div className="size-24 rounded-3xl bg-neutral-300" />
    <h2 className="text-2xl font-bold">{user.name}</h2>
  </section>
</div>
```

Bad Example (CSS Modules)

```
<div className={styles.outerContainer}>
  <section className={styles.innerSection}>
    <div className={styles.avatar} />
    <h2 className={styles.userName}>{user.name}</h2>
  </section>
</div>
```

Bad Example (Inline Styles)

```
<div style={{ margin: '0 auto', display: 'flex', width: '100%', maxWidth: '36rem',
flexDirection: 'column', gap: '1.5rem', padding: '1rem 1rem 1.5rem 1rem' }}>
  <section style={{ display: 'flex', width: '100%', flexDirection: 'column', alignItems:
'center', gap: '0.5rem', padding: '1.5rem' }}>
    <div style={{ width: '6rem', height: '6rem', borderRadius: '1.5rem', background:
'#d4d4d4' }} />
    <h2 style={{ fontSize: '1.5rem', fontWeight: 'bold' }}>{user.name}</h2>
  </section>
</div>
```

What's the problem in ALL these examples?

- Can't tell what these divs/sections represent
- Styling details overwhelm structure
- Not reusable
- Hard to review without CSS knowledge

The Solution: Semantic Component Extraction

Good Example (Works with ANY CSS approach)

```
<ProfileContainer>
  <ProfileHeader>
    <Avatar user={user} />
    <UserName>{user.name}</UserName>
  </ProfileHeader>
</ProfileContainer>
```

Benefits:

- Clear component hierarchy
- Communicates intent immediately
- Backend devs can understand structure
- Reusable across the app
- Styling details hidden in component files

The Four Core Principles

1. The Tech Lead Test

> "If someone who doesn't know your CSS framework can't understand your component hierarchy, you need better abstraction."

Ask yourself: Could a backend developer review this code and understand the page structure?

2. The Semantic Naming Test

> "If you can describe a component's purpose in 1-2 words, that should be its name."

```
// Bad: What is this?
<div className="...">

// Good: Immediately clear
<ProfileCard>
<NavigationMenu>
<PricingTable>
```

3. The DRY Test (Don't Repeat Yourself)

> "If you copy-paste the same styling pattern more than twice, extract a component."

4. The Single Level of Abstraction (SLA)

> "Keep all JSX at the same conceptual level. Don't mix high-level components with low-level HTML details."

Bad (mixed abstraction levels):

```
<ProfilePage>
  <Header />  {/* High-level component */}
  <div className="flex gap-4">  {/* Low-level HTML */}
    <div className="w-1/3 bg-gray-100 p-4">  {/* Low-level HTML */}
      <p className="font-bold">Name</p>
      <p>{user.name}</p>
    </div>
  </div>
  <Footer />  {/* High-level component */}
</ProfilePage>
```

Good (consistent abstraction level):

```
<ProfilePage>
  <Header />
  <ProfileContent>
    <ProfileSection title="Name" value={user.name} />
    <ProfileSection title="Email" value={user.email} />
  </ProfileContent>
  <Footer />
</ProfilePage>
```

Step-by-Step Tutorial: Component Extraction

Example: Refactoring a Profile Page

Step 1: Identify the Problem

Original Code (275 lines, Profile.tsx):

```
const Profile = () => {
  return (
    <div className="mx-auto flex w-full max-w-xl flex-col items-start gap-6 px-4 py-6">
      <section className="flex w-full flex-col items-center gap-2 p-6">
        <div className="size-24 rounded-3xl bg-neutral-300">
          <img src={user.avatarUrl} alt={user.name} />
        </div>
        <h2 className="text-2xl font-bold">{user.name}</h2>
        <div className="flex items-center">
          {[...Array(5)].map((_, i) => (
            <Star key={i} className="h-5 w-5 text-yellow-400" />
          ))}
        </div>
        <div className="flex items-center gap-4 text-sm text-gray-600">
          <span className="flex items-center gap-1.5">
            <Briefcase className="h-4 w-4" /> {user.role}
          </span>
          <span>• Joined {formatDate(user.joinedDate)}</span>
        </div>
        <div className="flex gap-2">
          <button className="rounded-full bg-neutral-200 px-4 py-2">Email</button>
          <button className="rounded-full bg-neutral-200 px-4 py-2">Message</button>
          <button className="rounded-full bg-neutral-200 px-4 py-2">Call</button>
        </button>
      </section>
      {/* ... 200+ more lines ... */}
    </div>
  );
};
```

Problems:

1. Everything in one file (275 lines)
2. No semantic component names
3. Can't understand structure without reading every line
4. Not reusable
5. Hard to test

Step 2: Identify Repeated Patterns

Ask yourself:

- What sections repeat?
- What could be reused elsewhere?
- What are the conceptual "things" on this page?

Pattern identification:

```
Profile Page
├── Container (layout wrapper)
├── Header
│   ├── Avatar
│   ├── Name
│   ├── Rating
│   ├── Metadata (role, joined date)
│   └── Action Buttons
├── About Section
├── Experience Section
│   └── Experience Entry (repeated)
├── Qualifications Section
├── Skills Section
└── Feedback Section
```

Step 3: Extract Layout Components

Create: `components/layout/ProfileContainer.tsx`

```tsx
import { ReactNode } from 'react';

interface ProfileContainerProps {
  children: ReactNode;
}

export const ProfileContainer = ({ children }: ProfileContainerProps) => {
  return (
    <div className="mx-auto flex w-full max-w-xl flex-col items-start gap-6 px-4 py-6">
      {children}
    </div>
  );
};
```

OR with CSS Modules:

```
// ProfileContainer.module.css
.container {
  margin: 0 auto;
  display: flex;
  width: 100%;
  max-width: 36rem;
  flex-direction: column;
  gap: 1.5rem;
  padding: 1rem;
}

// ProfileContainer.tsx
import styles from './ProfileContainer.module.css';

export const ProfileContainer = ({ children }) => {
  return <div className={styles.container}>{children}</div>;
};
```

Step 4: Extract Domain Components

Create: `components/profile/ProfileHeader.tsx`

```
import { Avatar } from '@/components/ui/Avatar';
import { StarRating } from '@/components/ui/StarRating';
import { ProfileActions } from './ProfileActions';
import type { UserProfile } from '@/types/user';

interface ProfileHeaderProps {
  user: UserProfile;
}

export const ProfileHeader = ({ user }: ProfileHeaderProps) => {
  return (
    <header className="flex w-full flex-col items-center gap-2 p-6">
      <Avatar src={user.avatarUrl} alt={user.name} size="large" />
      <h1 className="text-2xl font-bold">{user.name}</h1>
      <StarRating rating={user.rating} />
      <ProfileMetadata role={user.role} joinedDate={user.joinedDate} />
      <ProfileActions user={user} />
    </header>
  );
};
```

Benefits:

- Clear semantic name (`ProfileHeader`)
- All header logic in one place
- Reusable for similar profiles
- Easy to test in isolation

Step 5: Extract Reusable UI Components

Create: `components/ui/Avatar.tsx`

```
interface AvatarProps {
  src: string;
  alt: string;
  size?: 'small' | 'medium' | 'large';
}

export const Avatar = ({ src, alt, size = 'medium' }: AvatarProps) => {
  const sizeClasses = {
    small: 'size-12 rounded-2xl',
    medium: 'size-16 rounded-2xl',
    large: 'size-24 rounded-3xl'
  };

  return (
    <div className={sizeClasses[size]}>
      <img src={src} alt={alt} className="h-full w-full object-cover" />
    </div>
  );
};
```

Now reusable everywhere:

```
<Avatar src={user.avatar} alt={user.name} size="large" />
<Avatar src={commenter.avatar} alt={commenter.name} size="small" />
```

Step 6: Extract Repeated List Items

Create: `components/profile/ExperienceEntry.tsx`

```
import { MapPin, Ship, CalendarDays } from 'lucide-react';
import type { UserExperience } from '@/types/user';

interface ExperienceEntryProps {
  experience: UserExperience;
}

export const ExperienceEntry = ({ experience }: ExperienceEntryProps) => {
  return (
    <article className="flex items-center gap-4 border-b border-gray-100 py-4">
      <div className="size-24 rounded-3xl bg-neutral-300" />
      <div className="flex flex-col gap-1.5">
        <h4 className="font-semibold">{experience.title}</h4>
        <p className="flex items-center gap-2 text-sm text-gray-600">
          <MapPin className="h-4 w-4" />
          {experience.location}
        </p>
        <p className="flex items-center gap-2 text-sm text-gray-600">
          <Ship className="h-4 w-4" />
          {experience.vessel}
        </p>
        <p className="flex items-center gap-2 text-sm text-gray-600">
          <CalendarDays className="h-4 w-4" />
          {format(new Date(experience.date), "do MMM yyyy")}
        </p>
      </div>
    </article>
  );
};
```

Step 7: Assemble with Semantic Structure

Final: `pages/Profile.tsx`

```
import { ProfileContainer } from '@/components/layout/ProfileContainer';
import { ProfileHeader } from '@/components/profile/ProfileHeader';
import { ProfileSection } from '@/components/profile/ProfileSection';
import { ExperienceList } from '@/components/profile/ExperienceList';
import { QualificationsList } from '@/components/profile/QualificationsList';
import { FeedbackList } from '@/components/profile/FeedbackList';
import { useUserProfile } from '@/hooks/useUserProfile';

export const Profile = () => {
  const { user, loading, error } = useUserProfile();

  if (loading) return <LoadingSpinner />;
  if (error) return <ErrorMessage error={error} />;
  if (!user) return <NotFound />;

  return (
    <ProfileContainer>
      <ProfileHeader user={user} />

      <ProfileSection title="About & Hobbies">
        <p>{user.about}</p>
      </ProfileSection>

      <ProfileSection title="Experience Log">
        <ExperienceList experiences={user.experiences} />
      </ProfileSection>

      <ProfileSection title="Qualifications">
        <QualificationsList qualifications={user.qualifications} />
      </ProfileSection>

      <ProfileSection title="Skills">
        <p>{user.skills}</p>
      </ProfileSection>

      <ProfileSection title="Feedback">
        <FeedbackList feedback={user.feedback} />
      </ProfileSection>
    </ProfileContainer>
  );
};
```

Results:

- **60 lines** instead of 275 (78% reduction)
- **Immediately understandable** structure
- **Each component** < 100 lines
- **Reusable** components
- **Testable** in isolation
- **Reviewable** by anyone

Advanced Pattern: Polymorphic Components

Allow components to render different HTML elements for semantic correctness:

```
interface CardProps {
  children: ReactNode;
  element?: 'div' | 'article' | 'section' | 'aside';
}

export const Card = ({ children, element: Element = 'div' }: CardProps) => {
  return (
    <Element className="rounded-lg border p-4 shadow-sm">
      {children}
    </Element>
  );
};

// Usage - semantically correct everywhere
<Card element="article">
  <BlogPost />
</Card>

<Card element="aside">
  <Sidebar />
</Card>
```

Recommended File Structure

```
src/
├── components/
│   ├── layout/              # Layout containers
│   │   ├── ProfileContainer.tsx
│   │   ├── PageContainer.tsx
│   │   └── ContentWrapper.tsx
│   │
│   ├── ui/                  # Reusable UI primitives
│   │   ├── Avatar.tsx
│   │   ├── Button.tsx
│   │   ├── StarRating.tsx
│   │   └── Card.tsx
│   │
│   └── profile/             # Domain-specific components
│       ├── ProfileHeader.tsx
│       ├── ProfileSection.tsx
│       ├── ExperienceEntry.tsx
│       └── ProfileActions.tsx
│
└── pages/
    └── Profile.tsx          # Page composition
```

Checklist: When to Extract a Component

Extract when:

- ❑ A pattern appears 3+ times
- ❑ A section has clear responsibility (header, card, list item)
- ❑ JSX block exceeds 20-30 lines
- ❑ You can name it in 1-2 words
- ❑ It could be reused elsewhere
- ❑ It helps with Single Level of Abstraction

Don't extract when:

- ❑ It's only used once and simple (< 10 lines)
- ❑ The abstraction makes code harder to understand
- ❑ It requires too many props (> 8)

Practice Exercise

Given this code, extract semantic components:

```
const JobPage = () => {
  return (
    <div className="px-24 py-6">
      <div className="flex flex-col gap-4">
        <div className="flex flex-row gap-2.5">
          <div className="relative size-24 rounded-3xl bg-neutral-300">
            <Heart className="absolute top-2.5 right-2.5 cursor-pointer" />
          </div>
          <div className="flex flex-col justify-between py-2">
            <h2 className="font-semibold">{job.title}</h2>
            <div className="flex items-center gap-x-4">
              <p className="flex items-center gap-1">
                <Clock className="h-3.5 w-3.5" /> {job.type}
              </p>
              <p className="flex items-center gap-1">
                <CalendarDays className="h-3.5 w-3.5" /> {job.date}
              </p>
            </div>
          </div>
          <div className="ml-auto flex flex-col gap-4">
            <button className="rounded-xl bg-neutral-200">Share</button>
            <button className="rounded-xl bg-green-500">Apply</button>
          </div>
        </div>
      </div>
    </div>
  );
};
```

Your task:

1. Identify semantic sections
2. Create component names
3. Extract components
4. Refactor the page

Target structure:

```
<JobPageContainer>
  <JobHeader job={job} />
</JobPageContainer>
```

Additional Resources

Articles

- Semantic JSX by Max Böck
- React Component Design Principles
- Single Level of Abstraction Principle

Best Practices

- Favor composition over configuration
- Use semantic HTML elements
- Keep components focused (Single Responsibility)
- Name components by their purpose, not appearance
- Make components testable and reusable

Key Takeaways

1. **Component names should communicate intent** - Not implementation details
2. **Structure over styling** - Code should be readable regardless of CSS approach
3. **One level of abstraction** - Don't mix high and low-level code
4. **DRY principle** - Extract repeated patterns
5. **Think in components** - Not in divs, styling, or CSS classes

Remember: The goal is readable, maintainable, reviewable code that anyone on the team can understand.

Something unclear in this section? Open an issue.